

Para Estudo

1. Revisão de Algoritmos

1.1 Saída de Dados (`print`)

O comando `print()` é usado para exibir mensagens ou o conteúdo de variáveis no console (terminal).

```
# Exemplo 1: Imprimindo texto (string)
```

```
print("Olá, mundo!")
```

```
# Exemplo 2: Imprimindo variáveis
```

```
nome = "Maria"
```

```
idade = 25
```

```
print(nome, idade)
```

```
# Exemplo 3: F-strings (Forma moderna de formatar)
```

```
# Coloque um 'f' antes das aspas e as variáveis entre {}
```

```
print(f"O nome da aluna é {nome} e sua idade é {idade}.")
```

Parâmetros Avançados (`sep` e `end`)

Por padrão, `print()` separa os argumentos por espaço (`sep=' '`) e termina com uma quebra de linha (`end='\n'`). Isso pode ser alterado:

```
# Exemplo 4: Alterando o separador (sep)
```

```
print("maçã", "banana", "uva", sep=", ")
```

```
# Saída: maçã, banana, uva
```

```
# Exemplo 5: Alterando o final (end)
```

```
print("Esta é a primeira linha...", end=" ")
```

```
print("...e esta continua na mesma linha.")  
# Saída: Esta é a primeira linha... ...e esta continua na mesma linha.
```

1.2 Variáveis e Tipos de Dados

Variáveis são "contêineres" para armazenar valores. Python é uma linguagem de tipagem dinâmica, o que significa que você não precisa declarar o tipo; ele é inferido automaticamente.

- **str (String):** Texto. Ex: "Olá", 'Python'
- **int (Inteiro):** Números inteiros. Ex: 10, -5, 2025
- **float (Ponto Flutuante):** Números com casas decimais. Ex: 1.5, -10.99
- **bool (Booleano):** Valor lógico. Ex: True, False (Sempre com 'T' e 'F' maiúsculos)

```
nome_aluno = "João"      # str  
media_final = 9.5        # float  
idade = 19               # int  
aprovado = True          # bool
```

1.3 Operadores

Aritméticos

- + (Adição)
- - (Subtração)
- * (Multiplicação)
- / (Divisão - sempre retorna float)
- // (Divisão Inteira - descarta a parte decimal)
- % (Módulo - resto da divisão)
- ** (Exponenciação - elevado à potência)

```
print(10 / 3)    # Saída: 3.333...  
print(10 // 3)   # Saída: 3  
print(10 % 3)    # Saída: 1 (resto)  
print(2 ** 3)    # Saída: 8 (2*2*2)
```

Comparação (Retornam True ou False)

- == (Igual a)

- `!=` (Diferente de)
- `>` (Maior que)
- `<` (Menor que)
- `>=` (Maior ou igual a)
- `<=` (Menor ou igual a)

Lógicos (Combinam expressões booleanas)

- `and` (E lógico - Ambas as condições devem ser `True`)
- `or` (OU lógico - Pelo menos uma condição deve ser `True`)
- `not` (NÃO lógico - Inverte o valor)

```
idade = 20
possui_cnh = False

# Exemplo de 'and'
pode_dirigir = (idade >= 18) and (possui_cnh == True)
print(pode_dirigir) # Saída: False
```

1.4 Entrada de Dados (`input`)

O comando `input()` pausa o programa e espera o usuário digitar algo. **Crucial:** `input()` SEMPRE retorna o valor como uma **string** (`str`).

Você deve converter o tipo manualmente usando `int()` , `float()` , etc.

```
# 1. Pede o nome (str)
nome = input("Digite seu nome: ")

# 2. Pede a idade (str) e converte para int
idade_str = input("Digite sua idade: ")
idade_int = int(idade_str)

# 3. Forma direta (mais comum)
nota = float(input("Digite sua nota: "))

# 4. Cálculo
```

```
proxima_idade = idade_int + 1
print(f"Olá, {nome}. Ano que vem você terá {proxima_idade} anos.")
```

2. Estruturas de Controle

2.1 Condicionais (`if` / `elif` / `else`)

Executa blocos de código diferentes com base em uma condição ser `True` ou `False`. A indentação (espaços no início da linha) é obrigatória e define o bloco.

```
media = float(input("Digite a média final: "))

if media >= 7.0:
    print("Aprovado!")
    print("Parabéns!")
elif media >= 4.0:
    print("Recuperação.")
else:
    print("Reprovado.")

print("Fim do programa.") # Este 'print' está fora do bloco, sempre executa
```



2.2 Laço (`while`)

O `while` (enquanto) repete um bloco de código **enquanto** uma condição for verdadeira. É ideal para quando não sabemos quantas vezes vamos repetir.

Exemplo 1: Contador Simples

```
cont = 0
while cont < 5:
    print(f"O número é {cont}")
    cont = cont + 1 # IMPORTANTE: Atualizar a variável de controle

print("Laço 'while' encerrado.")
```

Exemplo 2: Condição de Parada (Flag)

```
numero = -1 # Inicializa com um valor que entra no laço

while numero != 0:
    numero = int(input("Digite um número (ou 0 para sair): "))
    print(f"Você digitou {numero}")

print("Você digitou 0. Fim.")
```

2.3 Laço (for)

O `for` (para) é usado para iterar sobre uma sequência (como uma lista, string ou um intervalo de números).

Exemplo 1: Iterando sobre uma Lista

```
frutas = ["Maçã", "Banana", "Uva"]

for fruta in frutas:
    print(f"Eu gosto de {fruta}")
```

Exemplo 2: Usando `range()`

A função `range()` gera uma sequência de números. É a forma mais comum de repetir algo N vezes.

```
# range(5) gera 0, 1, 2, 3, 4
for i in range(5):
    print(f"Repetição número {i}")

# range(1, 6) gera 1, 2, 3, 4, 5 (começa em 1, para ANTES do 6)
for i in range(1, 6):
    print(f"Contando: {i}")
```

2.4 Controle de Laços (`break` e `continue`)

Às vezes, precisamos de mais controle sobre a execução de um laço (`while` ou `for`).

- **break** : Interrompe e encerra o laço *imediatamente*. O código continua após o bloco do laço. (Visto no Exercício 1).
- **continue** : Interrompe a iteração *atual* e pula para o início da *próxima* iteração do laço.

```
# Exemplo de 'continue' (imprimir apenas números ímpares)
for i in range(1, 11): # Números de 1 a 10
    if i % 2 == 0: # Se o número for par...
        continue # ...pule esta iteração (o 'print' abaixo não executa)
    print(i) # Saída: 1, 3, 5, 7, 9

# Exemplo de 'break' (encontrar o primeiro número divisível por 7)
for i in range(20, 100):
    if i % 7 == 0:
        print(f"Encontrei o primeiro: {i}")
        break # Para o laço assim que achar
# Saída: Encontrei o primeiro: 21
```

3. Estruturas de Dados

3.1 Listas (Vetores Unidimensionais)

Listas são coleções ordenadas e **mutáveis** de itens. Elas são o equivalente a "vetores" ou "arrays" em outras linguagens. São definidas por colchetes `[]`.

```
# 1. Criando uma lista
notas = [9.5, 7.0, 8.5, 10.0]
nomes = ["Ana", "Bruno", "Carla"]
lista_vazia = []

# 2. Acessando itens (índice começa em 0)
print(nomes[0]) # Saída: Ana
print(notas[2]) # Saída: 8.5
print(nomes[-1]) # Saída: Carla (Índice negativo pega do final)

# 3. Alterando um item
nomes[1] = "Bruno Costa"
```

```
# 4. Adicionando um item (no final)
nomes.append("Daniel")

# 5. Removendo um item
nomes.pop()          # Remove o último (Daniel)
nomes.remove("Ana")  # Remove pelo valor "Ana"

# 6. Tamanho da lista
tamanho = len(nomes)
print(f"A lista tem {tamanho} nomes.")

# 7. Iterando com 'for' (muito comum)
for nota in notas:
    print(nota)
```

Fatiamento (Slicing)

É possível pegar "fatias" (sub-listas) de uma lista usando a sintaxe `lista[inicio:fim:passo]`.

```
numeros = [10, 20, 30, 40, 50, 60]

# Pega do índice 1 (20) ATÉ o índice 4 (que é o 50), sem incluí-lo
fatia1 = numeros[1:4]
print(fatia1) # Saída: [20, 30, 40]

# Pega do início ATÉ o índice 3 (sem incluí-lo)
fatia2 = numeros[:3]
print(fatia2) # Saída: [10, 20, 30]

# Pega do índice 2 ATÉ o final
fatia3 = numeros[2:]
print(fatia3) # Saída: [30, 40, 50, 60]
```

3.2 Listas (Matrizes Bidimensionais)

Matrizes em Python são simplesmente "listas dentro de listas".

```
# Exemplo do PDF (Cidades do Tocantins)
cidades = [
    ["Paraiso", 1963], # Linha 0
    ["Gurupi", 1958],  # Linha 1
```

```

    ["Palmas", 1989],    # Linha 2
    ["Miranorte", 1964] # Linha 3
]

# Acessando dados ( [linha][coluna] )
print(cidades[2][0])    # Saída: Palmas (Linha 2, Coluna 0)
print(cidades[0][1])    # Saída: 1963 (Linha 0, Coluna 1)

# Iterando sobre a matriz com 'for'
for cidade_info in cidades:
    # cidade_info é uma lista, ex: ["Paraiso", 1963]
    nome = cidade_info[0]
    ano = cidade_info[1]
    print(f"A cidade de {nome} foi fundada em {ano}.")

# Desempacotamento direto (mais "Pythônico")
for nome, ano in cidades:
    print(f"{nome} ({ano})")

```

3.3 Dicionários (`dict`)

Dicionários são coleções **não ordenadas** (em versões antigas do Python) de pares **chave-valor**. Eles são definidos por chaves `{ }`. Ideais para armazenar dados de um objeto (ex: aluno, produto).

```

# 1. Criando um dicionário
aluno = {
    "nome": "Carla",
    "matricula": "202403",
    "nota_final": 9.8,
    "ativo": True
}

# 2. Acessando valores (pela CHAVE)
print(aluno["nome"])    # Saída: Carla
print(aluno["nota_final"]) # Saída: 9.8
# Forma mais segura (não dá erro se a chave não existir)
print(aluno.get("curso", "Não informado")) # Saída: Não informado

# 3. Adicionando ou Alterando valores
aluno["nota_final"] = 10.0 # Altera valor existente
aluno["curso"] = "Sistemas de Informação" # Adiciona novo par

```

```
# 4. Iterando sobre um dicionário
for chave, valor in aluno.items():
    print(f"{chave}: {valor}")

# Saída:
# nome: Carla
# matricula: 202403
# ...etc
```

3.4 Tuplas (**tuple**)

Tuplas são coleções ordenadas e **imutáveis**. Uma vez criadas, não podem ser alteradas (não pode adicionar, remover ou modificar itens). São definidas por parênteses `()`.

São úteis para dados que não devem mudar, como coordenadas geográficas, datas ou chaves de dicionário compostas.

```
# 1. Criando uma tupla
coordenadas = (10.183, -48.333) # (Latitude, Longitude de Palmas)
dias_semana = ("Seg", "Ter", "Qua", "Qui", "Sex", "Sab", "Dom")

# 2. Acessando valores (igual à lista)
print(dias_semana[0]) # Saída: Seg
print(coordenadas[1]) # Saída: -48.333

# 3. Tentativa de alteração (GERA ERRO!)
# dias_semana[0] = "Segunda" # TypeError: 'tuple' object does not support i

# 4. Desempacotamento (muito comum)
lat, lon = coordenadas
print(f"Latitude: {lat}")
```

<

>

3.5 Conjuntos (**set**)

Conjuntos são coleções **não ordenadas** de itens **únicos** (sem duplicatas). São definidos por chaves `{}`, mas sem pares chave-valor.

Seu principal uso é remover duplicatas de uma lista e realizar operações matemáticas de conjuntos (união, interseção, etc.).

```
# 1. Criando um set (duplicatas são removidas)
numeros = {1, 2, 3, 2, 3, 4, 5, 1}
print(numeros) # Saída: {1, 2, 3, 4, 5} (ordem pode variar)

# 2. Criando um set a partir de uma lista (remove duplicatas)
lista_compras = ["Maçã", "Banana", "Maçã", "Laranja", "Banana"]
lista_unica = list(set(lista_compras))
print(lista_unica) # Saída: ['Maçã', 'Banana', 'Laranja'] (ordem pode variar)

# 3. Adicionando e Removendo
numeros.add(6) # Adiciona o 6
numeros.add(2) # Tenta adicionar o 2 (nada acontece, já existe)
numeros.remove(1) # Remove o 1
print(numeros) # Saída: {2, 3, 4, 5, 6}

# 4. Verificando pertinência (muito rápido)
print(4 in numeros) # Saída: True
```

4. Funções (def)

Funções são blocos de código reutilizáveis que executam uma tarefa. Elas são definidas com a palavra-chave `def`.

- **Parâmetros:** Variáveis que a função *recebe*.
- **return:** O valor que a função *devolve*.

```
# 1. Definindo uma função simples
def saudacao(nome):
    print(f"Olá, {nome}! Bem-vindo(a).")

# 2. Definindo uma função que retorna um valor
def somar(a, b):
    resultado = a + b
    return resultado

# --- Programa Principal ---

# 3. Chamando (executando) as funções
```

```
saudacao("Ana") # Chama a função, que vai imprimir "Olá, Ana!..."

valor_soma = somar(10, 5) # Chama a função e armazena o retorno
print(f"O resultado da soma é {valor_soma}") # Saída: 15
```

4.1 Parâmetros Opcionais e Type Hints

Valores Padrão (Opcionais)

Você pode definir um valor padrão para um parâmetro, tornando-o opcional na chamada da função.

```
# 'saud' é um parâmetro opcional com valor padrão "Olá"
def saudacao_avancada(nome, saud="Olá"):
    print(f"{saud}, {nome}!")

# Chamando sem o parâmetro opcional
saudacao_avancada("Bruno") # Saída: Olá, Bruno!

# Chamando e especificando o parâmetro opcional
saudacao_avancada("Carla", saud="Oi") # Saída: Oi, Carla!
```

Type Hints (Dicas de Tipo)

Python permite "anotar" os tipos esperados para parâmetros e para o valor de retorno. **Isso não é obrigatório e não gera erro**, mas ajuda muito na leitura e documentação do código.

```
# (a: int, b: int) -> int
# Diz que 'a' e 'b' deveriam ser inteiros, e a função deve retornar um intei
def multiplicar(a: int, b: int) -> int:
    return a * b

# (nome: str) -> None
# Diz que 'nome' deveria ser string, e a função não retorna nada (None)
def imprimir_nome(nome: str) -> None:
    print(f"Nome: {nome}")

# (lista_notas: list) -> float
def calcular_media(lista_notas: list) -> float:
    if not lista_notas:
```

```
    return 0.0  
return sum(lista_notas) / len(lista_notas)
```

5. Manipulação de Arquivos (I/O)

Python pode ler e escrever em arquivos de texto (`.txt` , `.csv` , etc.) de forma simples. A forma mais segura de fazer isso é com o gerenciador de contexto `with open()` , que garante que o arquivo será fechado automaticamente, mesmo se ocorrer um erro.

Iremos explorar os modos de abertura, como navegar dentro de um arquivo e como lidar com dados estruturados (CSV/JSON).

Modos de Abertura Básicos (`'w'` , `'a'` , `'r'`)

Escrevendo (`'w'` - Write)

Cria um novo arquivo (ou **apaga totalmente** um existente) e escreve conteúdo nele. O "cursor" começa no início.

```
# 1. Escrevendo (Modo 'w' - apaga o anterior)  
with open("meu_arquivo.txt", "w", encoding="utf-8") as arquivo:  
    arquivo.write("Esta é a primeira linha.\n")  
    arquivo.write("Esta é a segunda linha.\n")
```

Anexando (`'a'` - Append)

Adiciona conteúdo ao **final** de um arquivo existente, sem apagar o que já estava lá. Se o arquivo não existir, ele é criado.

```
# 2. Anexando (Modo 'a' - adiciona ao final)  
with open("meu_arquivo.txt", "a", encoding="utf-8") as arquivo:  
    arquivo.write("Esta linha foi anexada.\n")
```

Lendo (`'r'` - Read)

O **Modo** `'r'` é usado para ler o conteúdo de um arquivo. Este é o modo padrão (se você omitir o modo, Python usa `'r'`). Gera um erro (`FileNotFoundError`) se o arquivo não existir.

```
# 1. Tenta abrir o arquivo para leitura
try:
    with open("meu_arquivo.txt", "r", encoding="utf-8") as arquivo:
        # Exemplo A: Ler tudo de uma vez
        conteudo_completo = arquivo.read()
        print("--- Conteúdo Completo ---")
        print(conteudo_completo)

        # Para ler linhas (precisa 'voltar' ao início do arquivo)
        arquivo.seek(0)

        # Exemplo B: Ler linha por linha (ideal para arquivos grandes)
        print("\n--- Lendo Linha por Linha ---")
        for linha in arquivo:
            print(linha.strip()) # .strip() remove o '\n' do final

except FileNotFoundError:
    print("ERRO: O arquivo 'meu_arquivo.txt' não foi encontrado.")
except Exception as e:
    print(f"Ocorreu um erro inesperado: {e}")
```

5.1 Modos de Abertura Avançados (+ e b)

Podemos combinar os modos básicos para obter mais controle.

Modos de Leitura e Escrita (+)

O + permite que o arquivo seja lido E escrito.

- **r+ (Leitura + Escrita):** Abre um arquivo existente para ler e escrever. O cursor começa no início. (Gera erro se o arquivo não existir).
- **w+ (Escrita + Leitura):** Cria/Trunca um arquivo para escrever e ler. O cursor começa no início.
- **a+ (Anexar + Leitura):** Abre um arquivo para anexar (escrita começa no final) e ler. O cursor de *leitura* começa no início.

```
# Exemplo de 'r+' (ler e depois sobrescrever o início)
with open("meu_arquivo.txt", "r+", encoding="utf-8") as f:
    # 1. Lê o conteúdo
    conteudo = f.read()
    print(f"Conteúdo original: {conteudo.strip()}")
```

```
# 2. Volta ao início (veremos 'seek' em breve)
f.seek(0)

# 3. Escreve por cima do início
f.write("[SOBRESCRITO]")

# O resto do arquivo original pode permanecer, dependendo do tamanho da
```

Modo Binário (**b**)

Até agora, lidamos com arquivos de texto (usando `encoding="utf-8"`). Para manipular arquivos que não são texto (como imagens, PDFs, arquivos `.exe`, etc.), você DEVE usar o modo binário, adicionando `b`.

Em modo binário, você lê e escreve `bytes`, não `str`.

- `rb`: Ler binário.
- `wb`: Escrever binário.

```
# Exemplo de cópia de uma imagem (leitura binária + escrita binária)
try:
    with open("logo.png", "rb") as original:
        dados_binarios = original.read() # dados_binarios é do tipo 'bytes'

    with open("copia_logo.png", "wb") as copia:
        copia.write(dados_binarios)

    print("Imagem copiada com sucesso!")
except FileNotFoundError:
    print("Erro: Arquivo 'logo.png' não encontrado para cópia.")
```

5.2 Navegando no Arquivo (**seek** e **tell**)

Ao abrir um arquivo, o Python mantém um "cursor" (ponteiro) que indica onde a próxima operação de leitura ou escrita ocorrerá.

- `.tell()`: Retorna a posição atual (em bytes) do cursor no arquivo.
- `.seek(posicao)`: Move o cursor para a `posicao` (em bytes) especificada.

```
with open("meu_arquivo.txt", "r", encoding="utf-8") as f:
    # 1. Posição inicial
    print(f.tell()) # Saída: 0 (início)

    # 2. Lê os 10 primeiros caracteres (bytes, em UTF-8)
    primeiros_10 = f.read(10)
    print(primeiros_10)

    # 3. Verifica a nova posição
    print(f.tell()) # Saída: 10 (ou mais, dependendo do encoding)

    # 4. Volta ao início do arquivo
    f.seek(0)
    print(f.tell()) # Saída: 0

    # 5. Lê tudo de novo
    tudo = f.read()
    print("Lendo tudo:", tudo.strip())
```

5.3 Lidando com Dados Estruturados (CSV e JSON)

Embora possamos ler e escrever arquivos `.txt` manualmente (como no Exercício 5), para dados estruturados (como listas ou dicionários) é melhor usar os módulos padrões do Python.

Módulo `json` (Para Listas e Dicionários)

JSON é um formato excelente para salvar estruturas de dados complexas.

```
import json

# Nossos dados (uma lista de dicionários)
alunos = [
    {"nome": "Ana", "matricula": "1001"},
    {"nome": "Bruno", "matricula": "1002"}
]

# 1. Salvando em JSON (modo 'w')
try:
    with open("alunos.json", "w", encoding="utf-8") as f:
        json.dump(alunos, f, indent=4) # 'indent=4' formata o arquivo
    print("Arquivo 'alunos.json' salvo.")
```

```
# 2. Lendo de JSON (modo 'r')
with open("alunos.json", "r", encoding="utf-8") as f:
    dados_lidos = json.load(f)
    print("Dados lidos do JSON:")
    print(dados_lidos[0]["nome"]) # Saída: Ana
except Exception as e:
    print(f"Erro ao lidar com JSON: {e}")
```

Módulo `csv` (Para "Matrizes" / Planilhas)

CSV (Comma-Separated Values) é um formato de planilha simples.

```
import csv

cidades = [
    ["Paraiso", 1963],
    ["Gurupi", 1958]
]

# 1. Salvando em CSV (modo 'w')
try:
    with open("cidades.csv", "w", newline='', encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Cidade", "Ano"]) # Escreve o cabeçalho
        writer.writerows(cidades)         # Escreve todos os dados
    print("Arquivo 'cidades.csv' salvo.")

# 2. Lendo de CSV (modo 'r')
with open("cidades.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader) # Pula a linha do cabeçalho
    print("Dados lidos do CSV:")
    for linha in reader:
        print(f" - {linha[0]} (Ano: {linha[1]})")
except Exception as e:
    print(f"Erro ao lidar com CSV: {e}")
```

6. Gerenciando Caminhos (`pathlib`)

Lidar com arquivos envolve mais do que apenas ler e escrever; envolve gerenciar onde eles estão localizados. Lidar com caminhos de arquivos (ex: "C:\Usuarios\..." ou "/home/...") pode ser complicado.

O módulo `pathlib` (incluído no Python 3.4+) é a forma moderna e orientada a objetos de lidar com caminhos de sistema de arquivos.

```
from pathlib import Path

# 1. Importe a classe 'Path'
# (Opcional) 'Path.cwd()' pega o diretório atual de trabalho (onde o script
diretorio_atual = Path.cwd()
print(f"Estou rodando em: {diretorio_atual}")

# 2. Criando um objeto Path para um arquivo
# Note que o 'pathlib' usa a barra '/' mesmo no Windows
caminho_arquivo = Path("meu_arquivo.txt")
caminho_json = Path("dados/alunos.json") # Pode ser um subdiretório

# 3. Verificando informações do arquivo/caminho
print(f"O arquivo '{caminho_arquivo}' existe? {caminho_arquivo.exists()}")
print(f"O caminho '{caminho_json}' existe? {caminho_json.exists()}")

# 4. Criando diretórios
# Cria o diretório 'dados' se ele não existir
Path("dados").mkdir(exist_ok=True)

# 5. Usando 'pathlib' diretamente com 'open' (MUITO ÚTIL)
# Você não precisa converter para string, o 'open' entende o objeto Path
try:
    with open(caminho_json, "r", encoding="utf-8") as f:
        print("Lendo JSON via pathlib...")
except FileNotFoundError:
    print("O arquivo 'dados/alunos.json' ainda não foi criado.")

# 6. Listando arquivos em um diretório (Glob)
print("\nListando todos os arquivos .txt no diretório atual:")
# '*.txt' é um padrão "glob" (curinga)
for arquivo_txt in diretorio_atual.glob("*.txt"):
```

```
print(f" - {arquivo_txt.name}") # .name dá apenas o nome do arquivo

# 7. Lendo e escrevendo texto rapidamente (Python 3.5+)
try:
    caminho_arquivo.write_text("Conteúdo rápido!", encoding="utf-8")
    conteudo = caminho_arquivo.read_text(encoding="utf-8")
    print(f"\nLido com read_text(): {conteudo}")
except Exception as e:
    print(f"Erro com read/write_text: {e}")
```

7. Exercícios Resolvidos

Exercício 1: Média com **while** (Condição de Parada/Flag)

Crie um programa que solicite ao usuário a digitação de números inteiros. O programa deve parar quando o usuário digitar 0 (zero). No final, o programa deve exibir a média dos números digitados (excluindo o zero).

```
# Inicializa as variáveis de controle
soma_total = 0
contador = 0

print("Digite números para calcular a média (Digite 0 para sair):")

while True:
    try:
        numero = int(input(f"Digite o {contador + 1}º número: "))
    except:
        print("Valor inválido. Digite um número inteiro.")
        continue # Pula para a próxima iteração

    # Verifica a condição de parada (flag)
    if numero == 0:
        break # Encerra o laço 'while'

    soma_total += numero
    contador += 1
```

```
if contador > 0:
    media = soma_total / contador
    print(f"\nA média dos {contador} números foi: {media}")
else:
    print("\nNenhum número foi digitado (além do 0).")
```

Exercício 2: Iterando em Listas com `for` (Maior Valor)

Considere uma lista de compras, onde cada item é uma sub-lista contendo o nome do cliente e o valor da compra. Crie um programa que encontre e imprima o nome do cliente que fez a maior compra e o valor dela.

```
compras = [
    ["Ana", 150.75],
    ["Bruno", 520.00],
    ["Carla", 95.50],
    ["Daniel", 890.99],
    ["Elisa", 320.00]
]

if not compras: # Verifica se a lista não está vazia
    print("Lista de compras vazia.")
else:
    # Inicializa com o primeiro item
    maior_valor = compras[0][1]
    cliente_maior_compra = compras[0][0]

    for nome_cliente, valor_compra in compras:
        if valor_compra > maior_valor:
            maior_valor = valor_compra
            cliente_maior_compra = nome_cliente

    print(f"Valor da maior compra: R$ {maior_valor}; cliente: {cliente_maior_compra}")
```

Exercício 3: Funções e Matrizes (Cidades do Tocantins)

(Baseado no Exercício 3 da pág. 91 do PDF) Crie um programa com um menu interativo que utilize três funções: `imprimir_cidades()`, `verificar_data(ano)` e `acrescentar_cidade()`, usando a lista de cidades global.

```
# Variável global (geralmente evitada, mas usada aqui pelo enunciado)
cidades = [
    ["Paraíso", 1963], ["Gurupi", 1958], ["Palmas", 1989], ["Miranorte",
]

def imprimir_cidades():
    print("\n--- Lista de Cidades ---")
    for nome, ano in cidades:
        print(f"Cidade: {nome}; Ano de criação: {ano}")

def verificar_data(ano_usuario):
    print(f"\n--- Verificando ano: {ano_usuario} ---")
    for nome, ano_criacao in cidades:
        if ano_criacao < ano_usuario:
            print(f"A cidade de {nome} foi criada ANTES de {ano_usuario}")
        elif ano_criacao > ano_usuario:
            print(f"A cidade de {nome} foi criada DEPOIS de {ano_usuario}")
        else:
            print(f"A cidade de {nome} foi criada EXATAMENTE em {ano_usuario}")

def acrescentar_cidade():
    print("\n--- Adicionar Nova Cidade ---")
    nome = input("Digite o nome da nova cidade: ")
    try:
        ano = int(input(f"Digite o ano de fundação de {nome}: "))
        cidades.append([nome, ano])
        print(f"Cidade {nome} ({ano}) adicionada com sucesso!")
    except:
        print("Ano inválido. Operação cancelada.")

# --- Programa Principal (Menu) ---
while True:
    print("\n--- MENU CIDADES (1: Listar, 2: Verificar Ano, 3: Adicionar) ---")
    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        imprimir_cidades()
    elif opcao == "2":
        verificar_data(int(input("Digite o ano a ser verificado: ")))
    elif opcao == "3":
        acrescentar_cidade()
    else:
        print("Opção inválida. Tente novamente.")
```

```
    try:
        ano_teste = int(input("Digite o ano para comparar: "))
        verificar_data(ano_teste)
    except:
        print("Ano inválido.")
elif opcao == "3":
    acrescentar_cidade()
elif opcao == "0":
    print("Saindo...")
    break
else:
    print("Opção inválida.")
```

Exercício 4: Dicionário Alinhado (Lista de Dicionários)

Crie um sistema de cadastro de alunos. O sistema deve armazenar os alunos em uma **lista de dicionários**. Cada aluno (dicionário) deve ter: "nome", "matricula" e "notas" (uma lista de floats). Crie funções para:

1. Adicionar um novo aluno.
2. Listar todos os alunos.
3. Calcular e exibir a média de um aluno específico (buscando pela matrícula).

Estrutura de dados principal: Lista de Dicionários

```
cadastro_alunos = [
    {
        "nome": "Ana Silva",
        "matricula": "1001",
        "notas": [8.5, 9.0, 10.0]
    },
    {
        "nome": "Bruno Costa",
        "matricula": "1002",
        "notas": [6.0, 7.5, 5.0]
    }
]

def adicionar_aluno():
    print("\n--- Adicionar Aluno ---")
    nome = input("Nome do aluno: ")
```

```
matricula = input("Matrícula: ")

# Validação de notas
notas = []
try:
    n1 = float(input("Nota 1: "))
    n2 = float(input("Nota 2: "))
    n3 = float(input("Nota 3: "))
    notas = [n1, n2, n3]
except:
    print("Erro nas notas. Operação cancelada.")
    return # Sai da função

# Cria o novo dicionário do aluno
novo_aluno = {
    "nome": nome,
    "matricula": matricula,
    "notas": notas
}

# Adiciona o dicionário à lista principal
cadastro_alunos.append(novo_aluno)
print(f"Aluno {nome} adicionado com sucesso!")

def listar_alunos():
    print("\n--- Lista de Alunos ---")
    if not cadastro_alunos:
        print("Nenhum aluno cadastrado.")
        return

    for aluno in cadastro_alunos:
        # Itera sobre a lista de dicionários
        print(f"Nome: {aluno['nome']} (Mat: {aluno['matricula']})")
        print(f"  Notas: {aluno['notas']}")

def calcular_media_aluno():
    print("\n--- Calcular Média ---")
    matricula_busca = input("Digite a matrícula do aluno: ")

    aluno_encontrado = None
    # Procura o aluno na lista
    for aluno in cadastro_alunos:
        if aluno["matricula"] == matricula_busca:
```

```
        aluno_encontrado = aluno
        break # Para o 'for' assim que achar

if aluno_encontrado:
    nome = aluno_encontrado["nome"]
    notas = aluno_encontrado["notas"] # Pega a lista de notas

    if len(notas) > 0:
        media = sum(notas) / len(notas)
        print(f"A média do aluno {nome} é: {media:.2f}")
    else:
        print(f"O aluno {nome} não possui notas cadastradas.")
else:
    print(f"Erro: Aluno com matrícula {matricula_busca} não encontrado")

# --- Programa Principal (Menu) ---
while True:
    print("\n--- CADASTRO (1: Listar, 2: Adicionar, 3: Média, 0: Sair) ---")
    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        listar_alunos()
    elif opcao == "2":
        adicionar_aluno()
    elif opcao == "3":
        calcular_media_aluno()
    elif opcao == "0":
        print("Saindo...")
        break
    else:
        print("Opção inválida.")
```

Exercício 5: Manipulação de Arquivo (Salvar e Ler)

Crie um programa com duas funções:

1. `salvar_cidades_arquivo(lista_cidades)` : Recebe uma lista (matriz) de cidades e a salva em um arquivo `cidades.txt`. Cada sub-lista deve virar uma linha no arquivo, com os dados separados por vírgula (formato CSV simples).

2. `ler_cidades_arquivo()` : Lê o arquivo `cidades.txt`, o reconstrói na memória e o imprime no console.

```
# Dados iniciais
cidades_para_salvar = [
    ["Paraíso", 1963],
    ["Gurupi", 1958],
    ["Palmas", 1989]
]

NOME_ARQUIVO = "cidades_tocantins.txt"

def salvar_cidades_arquivo(lista_cidades: list):
    print(f"\nTentando salvar {len(lista_cidades)} cidades em '{NOME_ARQUIVO}'")
    try:
        with open(NOME_ARQUIVO, "w", encoding="utf-8") as f:
            for nome, ano in lista_cidades:
                # Formata a linha como: "Nome,Ano\n"
                linha = f"{nome},{ano}\n"
                f.write(linha)
            print("Arquivo salvo com sucesso!")
    except Exception as e:
        print(f"Erro ao salvar o arquivo: {e}")

def ler_cidades_arquivo() -> list:
    print(f"\nLendo dados do arquivo '{NOME_ARQUIVO}'...")
    lista_lida = []
    try:
        with open(NOME_ARQUIVO, "r", encoding="utf-8") as f:
            for linha in f:
                # 1. Remove espaços/quebras de linha extras
                linha_limpa = linha.strip() # Ex: "Palmas,1989"

                if not linha_limpa: # Pula linhas em branco
                    continue

                # 2. Divide a linha pela vírgula
                partes = linha_limpa.split(",") # Ex: ["Palmas", "1989"]

                # 3. Reconstrói os dados (convertendo o ano para int)
                if len(partes) == 2: # Garante que a linha não está mal
                    nome = partes[0]
```

```

        ano = int(partes[1])
        lista_lida.append([nome, ano])

    # Imprime os dados lidos
    print("Dados lidos do arquivo:")
    for nome, ano in lista_lida:
        print(f" - {nome} ({ano})")
    return lista_lida

except FileNotFoundError:
    print(f"ERRO: O arquivo '{NOME_ARQUIVO}' não foi encontrado.")
except Exception as e:
    print(f"Erro ao ler o arquivo: {e}")
return [] # Retorna lista vazia em caso de erro

# --- Programa Principal ---
# 1. Salva os dados no arquivo
salvar_cidades_arquivo(cidades_para_salvar)

# 2. Lê os dados de volta do arquivo
dados_lidos = ler_cidades_arquivo()

```

Exercício 6: Funções, Sets e Tuplas (Limpeza de Dados)

Crie uma função `processar_lista(numeros)` que receba uma lista de inteiros (que pode conter duplicatas). A função deve remover todas as duplicatas e retornar uma **tupla** contendo os números únicos em **ordem crescente**.

```

def processar_lista(numeros: list) -> tuple:
    # 1. Converte a lista para um set (isso remove as duplicatas)
    numeros_unicos_set = set(numeros)

    # 2. Converte o set de volta para uma lista para poder ordenar
    lista_ordenada = sorted(list(numeros_unicos_set))

    # 3. Converte a lista ordenada para uma tupla e a retorna
    return tuple(lista_ordenada)

# --- Programa Principal ---

```

```
entrada = [1, 5, 2, 2, 8, 10, 5, 1, 3, 10]
print(f"Lista de entrada: {entrada}")

saida_tupla = processar_lista(entrada)

print(f"Tupla de saída (únicos e ordenados): {saida_tupla}")
# Saída: Tupla de saída (únicos e ordenados): (1, 2, 3, 5, 8, 10)
```

Exercício 7: Dicionários e Strings (Contador de Palavras)

Crie uma função `contar_palavras(texto)` que receba uma string (frase ou parágrafo). A função deve:

1. Converter todo o texto para minúsculas.
2. Remover pontuações simples (apenas `.` e `,`).
3. Retornar um **dicionário** onde cada chave é uma palavra e seu valor é o número de vezes que ela apareceu.

```
def contar_palavras(texto: str) -> dict:
    # 1. Converte para minúsculas
    texto_limpo = texto.lower()

    # 2. Remove pontuação
    texto_limpo = texto_limpo.replace(",", "")
    texto_limpo = texto_limpo.replace(".", "")

    # 3. Quebra a string em uma lista de palavras
    palavras = texto_limpo.split() # split() sem argumento divide por es

    # 4. Cria o dicionário de frequência
    frequencia = {}

    for palavra in palavras:
        # Pega o valor atual (ou 0 se a chave não existir) e soma 1
        frequencia[palavra] = frequencia.get(palavra, 0) + 1

    return frequencia

# --- Programa Principal ---
texto_exemplo = "Olá mundo, este é um teste. Olá de novo, mundo."
```

```
contagem = contar_palavras(texto_exemplo)

print(f"Texto: '{texto_exemplo}'")
print("Contagem de palavras:")
print(contagem)
# Saída:
# {'olá': 2, 'mundo': 2, 'este': 1, 'é': 1, 'um': 1, 'teste': 1, 'de': 1
```

Exercício 8: `pathlib` e Leitura (Consolidador de Relatórios)

Suponha que você tenha um diretório chamado `relatorios/` e dentro dele vários arquivos `.txt` (`dia1.txt`, `dia2.txt`, ...). Crie um programa que:

1. Use `pathlib` para garantir que o diretório `relatorios/` exista (e o crie, se necessário).
2. (Para fins de teste) Crie 2 arquivos de exemplo dentro dele (`dia1.txt` e `dia2.txt`).
3. Use `pathlib.glob()` para encontrar todos os arquivos `.txt` dentro de `relatorios/`.
4. Leia o conteúdo de cada um desses arquivos e o junte em um único arquivo chamado `relatorio_completo.txt`.

```
from pathlib import Path

# Define o nome do diretório e do arquivo final
DIRETORIO_RELATORIOS = Path("relatorios")
ARQUIVO_FINAL = Path("relatorio_completo.txt")

def preparar_ambiente_teste():
    print(f"1. Garantindo que o diretório '{DIRETORIO_RELATORIOS}' exista")
    DIRETORIO_RELATORIOS.mkdir(exist_ok=True)

    # Cria arquivos de exemplo
    (DIRETORIO_RELATORIOS / "dia1.txt").write_text("Relatório do Dia 1: ")
    (DIRETORIO_RELATORIOS / "dia2.txt").write_text("Relatório do Dia 2: ")
    (DIRETORIO_RELATORIOS / "config.ini").write_text("admin=true", encoding="utf-8")
    print("    - Ambiente de teste criado.")

def consolidar_relatorios():
    print("\n2. Iniciando consolidação...")

    # Procura por TODOS os arquivos .txt dentro do diretório
```

```
arquivos_txt = list(DIRETORIO_RELATORIOS.glob("*.txt"))

if not arquivos_txt:
    print("    - Nenhum arquivo .txt encontrado.")
    return

print(f"    - Encontrados {len(arquivos_txt)} arquivos .txt.")

# Abre o arquivo final no modo 'w' (para apagar o anterior)
try:
    with open(ARQUIVO_FINAL, "w", encoding="utf-8") as f_final:
        for caminho_arquivo in arquivos_txt:
            # Adiciona um cabeçalho para saber de onde veio o texto
            f_final.write(f"--- CONTEÚDO DE {caminho_arquivo.name} ---")

            # Lê o conteúdo do arquivo de relatório
            conteudo = caminho_arquivo.read_text(encoding="utf-8")

            # Escreve no arquivo consolidado
            f_final.write(conteudo)
            f_final.write("\n") # Adiciona espaço

        print(f"\n3. Sucesso! Relatório consolidado salvo em '{ARQUIVO_FINAL}'")
except Exception as e:
    print(f"Erro durante a consolidação: {e}")

# --- Programa Principal ---
preparar_ambiente_teste()
consolidar_relatorios()
```